

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Vasantha Nithi D Y	Reg. No.:	192424137
Section / Batch:	2024	Date:	

Code of Conduct

I Vasantha Nithi D Y (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3

Finite-State Morphological Analysis, RuleBased Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemmingbased preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove

prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1	3	4	4	3	4	18	90
2	4	4	3	3	4	19	
3	3	3	3	3	4	17	
4	4	4	4	3	4	18	
5	3	4	4	3	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____

Q1. Morphological Analysis of connected, connecting, connection

Manual Solution

Word	Root	Suffix Type	Normalized Form
connected	connect	-ed Inflectional	connect
connecting	connect	-ing Inflectional	connect
connection	connect	-ion Derivational	connect

Breakdown:

- connect + ed → connected
- connect + ing → connecting
- connect + ion → connection

-ed and -ing indicate grammatical forms, so they are **inflectional**.
-ion changes the verb *connect* into the noun *connection*, so it is **derivational**.

Python Program words = ["connected",
"connecting", "connection"]

```
for word in words:    if
word.endswith("ed"):
    root = word[:-2]
    suffix = "-ed"      type_
    = "Inflectional"

    elif word.endswith("ing"):
        root = word[:-3]
        suffix = "-ing"    type_
        = "Inflectional"

    elif word.endswith("ion"):
        root = word[:-3]
        suffix = "-ion"    type_ =
        "Derivational"
```

```
print(word, "\t", root, "\t", suffix, "\t", type_)
```

Output

```
connected  connect  -ed  Inflectional connecting
connect  -ing  Inflectional connection  connect
-ion  Derivational
```

Normalized output: connected, connecting, and connection → **connect**.

Q2. Morphological Parsing of unhappy, happiness, happily

Manual Solution

Word	Prefix	Base	Suffix	Type	Normalized Root
unhappy	un-	happy	—	Derivational	happy
happiness	—	happy	-ness	Derivational	happy
happily	—	happy	-ly	Derivational	happy

Breakdown:

- un- + happy → **unhappy** → un- gives a negative meaning.
- happy + -ness → **happiness** → -ness changes the adjective into a noun expressing a state/quality.
- happy + -ly → **happily** → -ly changes the adjective into an adverb.

All three are **derivational** because the affixes create new words and/or change their grammatical category.

Python Program words = ["unhappy",
"happiness", "happily"]

for word in words:

```
    if word.startswith("un"):
        prefix = "un-"
    base = word[2:]    suffix
    = "-"    type_ =
    "Derivational"
```

```
    elif word.endswith("ness"):
        prefix = "-"
```

```

        base = word[:-4]
suffix = "-ness"      type_
= "Derivational"

elif word.endswith("ly"):
    prefix = "-"      base
= word[:-2]      suffix = "-
ly"      type_ =
"Derivational"

print(word, "\t", prefix, "\t", base, "\t", suffix, "\t", type_)

```

Output

```

unhappy  un-  happy  -   Derivational
happiness -   happy -ness Derivational
happily  -   happy  -ly  Derivational

```

Normalized root: unhappy, happiness, and happily → **happy**.

Q3. Stemming of played, player, playing

Manual Solution

Word	Removed Affix	Stem	Type	Normalized Form
played	-ed	play	Inflectional	play
player	-er	play	Derivational	play
playing	-ing	play	Inflectional	play

Stemming:

- play + ed → **played** → remove -ed → **play**
- play + er → **player** → remove -er → **play**
- play + ing → **playing** → remove -ing → **play**

-ed and -ing represent grammatical variations, while -er forms a new noun (*player*), so it is derivational. All forms are normalized to **play** for search indexing.

Python Program

```

words = ["played", "player", "playing"]

for word in words:

    if word.endswith("ing"):
stem = word[:-3]
affix = "-ing"    type_ =
"Inflectional"

    elif word.endswith("ed"):
stem = word[:-2]    affix
= "-ed"    type_ =
"Inflectional"

    elif word.endswith("er"):
stem = word[:-2]    affix
= "-er"    type_ =
"Derivational"

print(word, "\t", stem, "\t", affix, "\t", type_)

```

Output

```

played  play  -ed  Inflectional
player  play  -er  Derivational
playing play  -ing Inflectional

```

Normalized form: all three → **play**.

Q4. Finite-State Morphological Parsing of writes, writing, written

Manual Solution

We use **write** as the common root.

Word Morphological Breakdown	Type	Normalized Form
writes	write + -s	Regular Inflectional write

writing write + -ing

Regular Inflectional **write**

written write + -ten

Irregular Inflectional **write**

State Transitions q0

--write--> q1

q1 --s-----> q2 → writes q1

--ing---> q3 → writing q1

--ten---> q4 → written

- writes → regular **-s** inflection.
- writing → regular **-ing** inflection.
- written → irregular past participle form; it cannot be obtained simply by adding -ed.

All three are normalized to **write**.

Python Program

```
words = ["writes", "writing", "written"]
```

```
for word in words:
```

```
    if word == "writes":
```

```
        root = "write"
```

```
        transition = "q0 -> q1 -> q2"
```

```
components = "write + s"      type_ =
```

```
"Regular Inflection"
```

```
    elif word == "writing":
```

```
        root = "write"
```

```
        transition = "q0 -> q1 -> q3"
```

```
components = "write + ing"    type_ =
```

```
"Regular Inflection"
```

```
    elif word == "written":
```

```
        root = "write"      transition
```

```
= "q0 -> q1 -> q4"
```

```
components = "write + ten"
type_ = "Irregular Inflection"
```

```
print(word, "\t", transition, "\t", components, "\t", root, "\t", type_)
```

Output

```
writes  q0 -> q1 -> q2  write + s    write  Regular Inflection writing
q0 -> q1 -> q3  write + ing   write  Regular Inflection written  q0 ->
q1 -> q4  write + ten   write  Irregular Inflection Normalized form:
all three → write.
```

Q5. Porter Stemmer Processing of relational, relation, relate

Manual Solution

Using Porter stemming rules:

Word	Stemming Steps	Final Stem
relational	relational → relate → relat	relat relation
	relation → relate → relat	relat
relate	relate → relat	relat

Stemming Breakdown

1. relational

relational ↓

remove -ion

relat **2. relation**

relation ↓

remove -ion

relat

3. relate relate

↓ remove final -e

relat

Thus, all three words are normalized to:

This follows the question's requirement to apply Porter stemming rules sequentially and show the intermediate/final forms.

Python Program from nltk.stem

```
import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
words = ["relational", "relation", "relate"]
```

```
print("Word\t\tStem")
```

```
for word in words:
```

```
    stem = stemmer.stem(word)
```

```
print(word, "\t", stem)
```

Output

Word	Stem
------	------

relational	relat
------------	-------

relation	relat relate
----------	--------------

relat	
-------	--

Final normalized stem for all three: relate.